SQL ASSIGNMENT 17

Subject: Database Management Systems (SQL) **Topic:** Aggregations, Joins, Subqueries, Window Functions & Index Optimization **Status:** Completed

TASK 1

Write a SQL query to display the total number of songs uploaded by each artist from the `songs` table (columns: `song_id`, `artist_name`, `title`), showing only artists who have uploaded more than 3 songs.

```
SELECT artist_name, COUNT(song_id) AS total_songs FROM songs GROUP BY artist_name
HAVING COUNT(song_id) > 3;
```

TASK 2

Given `orders` (order_id, user_id, amount) and `users` (user_id, username), write a SQL `JOIN` query to display each username along with their total order amount.

```
SELECT u.username, COALESCE(SUM(o.amount), 0) AS total_order_amount FROM users u LEFT
JOIN orders o ON u.user_id = o.user_id GROUP BY u.user_id, u.username;
```

TASK 3

Write a SQL subquery to find the names of all restaurants from a `restaurants` table (id, name, rating) whose rating is higher than the overall average rating.

```
SELECT name, rating FROM restaurants WHERE rating > ( SELECT AVG(rating) FROM
restaurants );
```

TASK 4

Using a `transactions` table (id, user_id, amount, transaction_date), write a window function query to display each user's transaction amount and running total ordered by date.

```
SELECT user_id, amount, transaction_date, SUM(amount) OVER ( PARTITION BY user_id ORDER
BY transaction_date ) AS running_total FROM transactions;
```

TASK 5

List two optimizations to speed up a query filtering Flipkart products by category and price, and explain how each helps.

- **1. Create a Composite Index on `(category, price)`:** A composite index allows the query planner to quickly jump directly to the target category and evaluate price range conditions via index range scans, avoiding expensive full table scans.
- **2. Selective Column Projection (Avoid `SELECT *`):** Querying only specific required columns (e.g., `id, name, price`) enables index-only scans if fields are covered, minimizing IO overhead and memory consumption.